PROJECT REPORT

생성형 AI RAG 시스템 이해와 개선

청킹(Chunking) 전략 튜닝을 통한 검색 정확도 개선 사례

RAG(Retrieval-Augmented Generation) 파이프라인 분석 보고서

01

생성형 AI API 호출 흐름에 대한 이해

사용자 질문이 답변으로 만들어지기까지, RAG는 5단계를 거칩니다.



핵심 포인트 RAG는 LLM을 재학습시키지 않습니다. 질문과 관련된 문서 조각을 찾아 프롬프트에 함께 넣어주는 방식입니다.

02

문서 업로드 전 / 후 AI 응답의 차이

예시 질문 — "아트빌리지 주차장 몇 개야?"

문서 업로드 전

일반 지식(사전학습 데이터)에만 의존

- 특정 장소·최신 정보는 모르거나 부정확
- 그럴듯하지만 틀린 답변(할루시네이션) 위험
- 출처를 제시할 수 없음

예상 답변: *"정확한 주차 대수는 확인이 어렵습니다" 또는 근거 없는 추측 응답*

문서 업로드 후 (RAG)

업로드된 실제 문서 기반으로 답변

- 관광지별 구체적 수치·정보까지 정확히 응답
- 문서에 근거한 답변으로 신뢰도 상승
- 근거 문서(출처) 추적 가능

예상 답변: *"아트빌리지 주차장은 총 OO대 규모입니다" (문서 수치 기반 응답)*

03

RAG 적용 시 장점과 한계

검색 기반 증강 생성은 강력하지만, 검색 품질에 답변 품질이 좌우됩니다.

장점

최신 정보 반영

모델 재학습 없이 새 문서만 추가하면 최신 지식 반영 가능

도메인 특화 지식

관광지 등 특정 분야 데이터를 즉시 주입해 전문성 확보

출처 추적 가능

답변의 근거가 된 문서를 함께 제시해 신뢰도 확보

한계

검색 품질 의존

검색이 부정확하면 답변도 부정확 (Garbage in, garbage out)

청킹 전략 민감

분할 방식이 부적절하면 문맥이 잘려 정보 유실 발생

비용·속도 트레이드오프

검색 청크가 많아질수록 응답 속도와 비용 부담 증가

04

데이터(문서)의 품질이 응답에 미치는 영향

청킹(Chunking) 전략 튜닝 사례 — 글자 수 절단 → 마크다운 구조 기반 분할

Before — 글자 수 기준 절단

CHUNK_SIZE=500, OVERLAP=50 (글자 수 기준, 구조 무시)

청크 A: 아트빌리지 소개, 위치…

청크 B: …주차장 정보

- 관광지 정보가 두 청크로 쪼개짐
- "아트빌리지 주차장 몇 개?" → 정보가 다른 청크에 있어 검색 실패

After — 구조 기반 청킹

마크다운 헤더(##) · 구분자(---) 기준으로 분할

청크 1개 = 아트빌리지 전체 정보 (소개·위치·주차장 포함)

- 관광지 단위 정보 무결성 유지
- 질문과 관련된 정보가 한 청크 안에 모두 존재 → 검색 정확도 향상

결과 마크다운 구조 기준 청킹으로 전환 후, 항목별 정보가 하나의 청크로 유지되어 검색 및 답변 정확도가 개선됨을 확인.

05

향후 개선 및 확장 가능성

청킹 튜닝을 시작으로, 검색 정확도를 더 끌어올릴 수 있는 다음 단계

메타데이터 태깅

관광지명·카테고리 등 항목별 메타데이터를 부여해 검색 정확도를 추가로 향상

하이브리드 검색

키워드 검색과 벡터 검색을 결합해 정확한 고유명사 검색까지 보완

동적 청크 크기

항목 길이에 따라 청크 크기를 유동적으로 조정하는 방식 도입

정량 평가 체계

질의응답 정확도를 측정하는 테스트셋을 구축해 개선 효과를 검증